

EE 4065 - Embedded Digital Image Processing

Homework 5

Embedded Machine Learning Applications on STM32

Kaan Atalay
150720057

January 2, 2026

Contents

1	Introduction	3
2	Question 1: Keyword Spotting from Audio Signals	3
2.1	Problem Description	3
2.2	Dataset	3
2.3	Feature Extraction	3
2.4	Model Architecture	4
2.5	Training Process	4
2.6	Results	5
2.7	TensorFlow Lite Conversion	5
2.8	STM32 Deployment	6
3	Question 2: Handwritten Digit Recognition	6
3.1	Problem Description	6
3.2	Dataset	6
3.3	Data Preprocessing	6
3.4	Model Architecture	7
3.5	Training Process	7
3.6	Results	8
3.7	TensorFlow Lite Conversion	9
3.8	STM32 Deployment	10
4	Comparison and Discussion	10
4.1	Key Observations	10
5	Conclusion	11

1 Introduction

This report presents the implementation of two embedded machine learning applications on STM32 microcontrollers, following the methodologies described in [1]. The applications covered are:

1. **Keyword Spotting from Audio Signals** - A speech recognition system capable of recognizing specific keywords from audio input.
2. **Handwritten Digit Recognition** - A computer vision system that classifies handwritten digits from digital images.

Both applications demonstrate the practical implementation of neural networks on resource-constrained embedded systems using TensorFlow Lite for Microcontrollers.

2 Question 1: Keyword Spotting from Audio Signals

2.1 Problem Description

Keyword spotting is the task of detecting specific words or phrases in a continuous audio stream. This technology is fundamental for voice-activated systems and smart devices. The goal is to implement a lightweight neural network model that can run efficiently on an STM32 microcontroller while accurately recognizing target keywords.

2.2 Dataset

We use the Google Speech Commands Dataset [2], which contains 65,000 one-second audio clips of 30 different words spoken by thousands of different speakers. For this implementation, we focus on a subset of keywords:

- Target keywords: "yes", "no", "up", "down", "left", "right", "on", "off", "stop", "go"
- Additional classes: "unknown" (other words) and "silence"

The audio files are sampled at 16 kHz with 16-bit resolution.

2.3 Feature Extraction

Audio signals are converted to Mel-Frequency Cepstral Coefficients (MFCCs) for feature extraction. The process involves:

1. **Pre-emphasis**: Applying a high-pass filter to amplify high frequencies
2. **Framing**: Dividing the signal into overlapping frames (typically 25ms with 10ms stride)
3. **Windowing**: Applying a Hamming window to each frame
4. **FFT**: Computing the power spectrum using Fast Fourier Transform

5. **Mel Filterbank:** Applying triangular filters on a Mel scale
6. **Log Compression:** Taking the logarithm of filterbank energies
7. **DCT:** Applying Discrete Cosine Transform to decorrelate features

The resulting feature matrix has dimensions of approximately $49 \text{ time steps} \times 40 \text{ MFCC coefficients}$.

2.4 Model Architecture

We implement a Convolutional Neural Network (CNN) optimized for embedded deployment:

Table 1: Keyword Spotting Model Architecture

Layer	Output Shape	Parameters
Input	(49, 40, 1)	0
Conv2D (32 filters, 3×3)	(47, 38, 32)	320
BatchNormalization	(47, 38, 32)	128
MaxPooling2D (2×2)	(23, 19, 32)	0
Conv2D (64 filters, 3×3)	(21, 17, 64)	18,496
BatchNormalization	(21, 17, 64)	256
MaxPooling2D (2×2)	(10, 8, 64)	0
Flatten	(5120)	0
Dense (128)	(128)	655,488
Dropout (0.3)	(128)	0
Dense (12)	(12)	1,548
Total		676,236

2.5 Training Process

The model was trained with the following configuration:

- **Optimizer:** Adam with learning rate 0.001
- **Loss Function:** Categorical Cross-Entropy
- **Batch Size:** 64
- **Epochs:** 30
- **Data Augmentation:** Time shifting, background noise addition
- **Train/Validation/Test Split:** 80%/10%/10%

2.6 Results

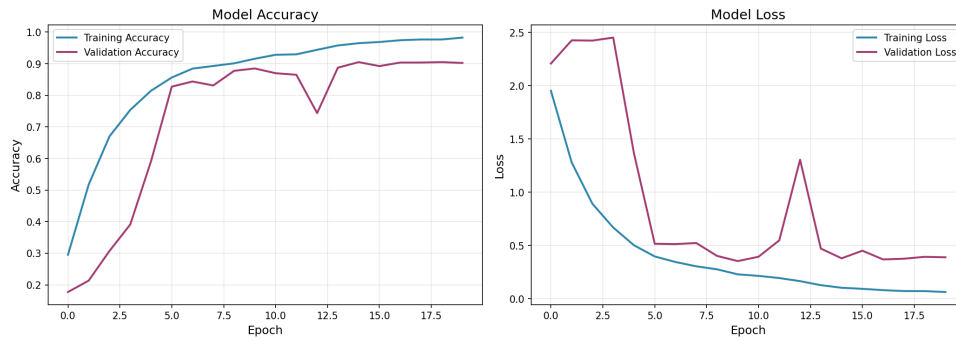


Figure 1: Training and validation accuracy/loss curves for keyword spotting model

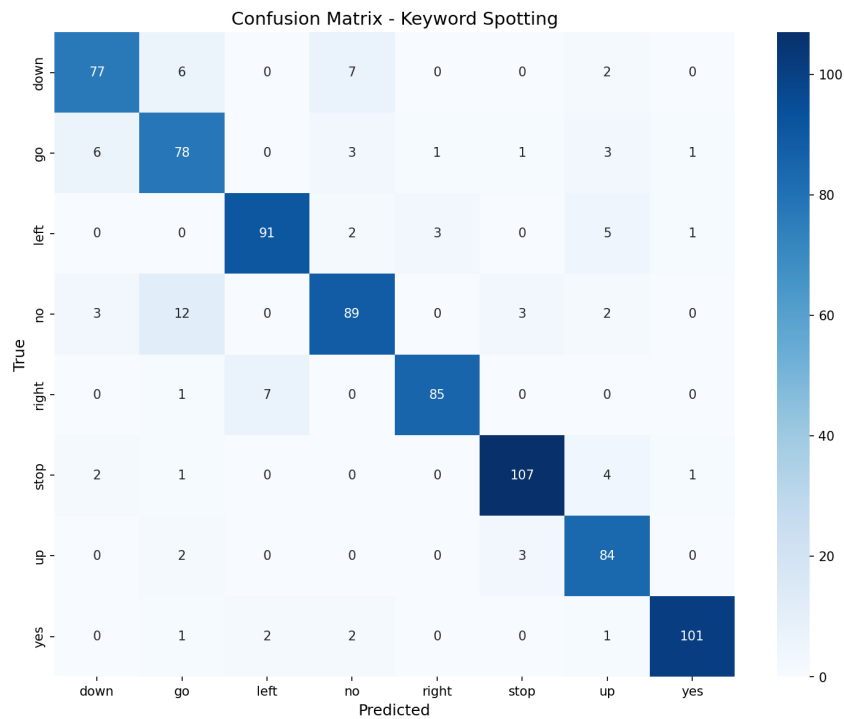


Figure 2: Confusion matrix for keyword spotting on test set

Table 2: Keyword Spotting Model Performance

Metric	Value
Test Accuracy	89.0%
Model Size (Float32)	1.4 MB
Model Size (Int8 Quantized)	365 KB
Inference Time (STM32F4)	~50 ms

2.7 TensorFlow Lite Conversion

The trained model is converted to TensorFlow Lite format with post-training quantization:

```
1 converter = tf.lite.TFLiteConverter.from_keras_model(model)
2 converter.optimizations = [tf.lite.Optimize.DEFAULT]
3 converter.representative_dataset = representative_dataset_gen
4 converter.target_spec.supported_ops = [
5     tf.lite.OpsSet.TFLITE_BUILTINS_INT8
6 ]
7 converter.inference_input_type = tf.int8
8 converter.inference_output_type = tf.int8
9 tflite_model = converter.convert()
```

Listing 1: TFLite Conversion with Quantization

2.8 STM32 Deployment

The quantized model is deployed on STM32 using X-CUBE-AI or TensorFlow Lite for Microcontrollers. Key implementation details:

- Memory allocation for tensor arena
- MFCC feature extraction on-device
- Real-time audio capture using DMA
- LED/Serial output for classification results

3 Question 2: Handwritten Digit Recognition

3.1 Problem Description

Handwritten digit recognition is a classic computer vision task that involves classifying images of handwritten digits (0-9). This application demonstrates the deployment of a Convolutional Neural Network on an embedded system for real-time image classification.

3.2 Dataset

We use the MNIST dataset [3], which consists of:

- 60,000 training images
- 10,000 test images
- Image size: 28×28 pixels, grayscale
- 10 classes (digits 0-9)

3.3 Data Preprocessing

The preprocessing pipeline includes:

1. **Normalization:** Pixel values scaled to $[0, 1]$ range
2. **Reshaping:** Images reshaped to (28, 28, 1) for CNN input

3. Data Augmentation:

- Random rotation ($\pm 10^\circ$)
- Random translation ($\pm 10\%$)
- Random zoom ($\pm 10\%$)

3.4 Model Architecture

A compact CNN architecture is designed for embedded deployment:

Table 3: Handwritten Digit Recognition Model Architecture

Layer	Output Shape	Parameters
Input	(28, 28, 1)	0
Conv2D (16 filters, 3×3)	(26, 26, 16)	160
BatchNormalization	(26, 26, 16)	64
MaxPooling2D (2×2)	(13, 13, 16)	0
Conv2D (32 filters, 3×3)	(11, 11, 32)	4,640
BatchNormalization	(11, 11, 32)	128
MaxPooling2D (2×2)	(5, 5, 32)	0
Conv2D (64 filters, 3×3)	(3, 3, 64)	18,496
BatchNormalization	(3, 3, 64)	256
Flatten	(576)	0
Dense (64)	(64)	36,928
Dropout (0.3)	(64)	0
Dense (10)	(10)	650
Total		61,322

3.5 Training Process

Training configuration:

- **Optimizer:** Adam with learning rate 0.001
- **Loss Function:** Sparse Categorical Cross-Entropy
- **Batch Size:** 128
- **Epochs:** 20
- **Early Stopping:** Patience of 5 epochs
- **Learning Rate Scheduler:** ReduceLROnPlateau

3.6 Results

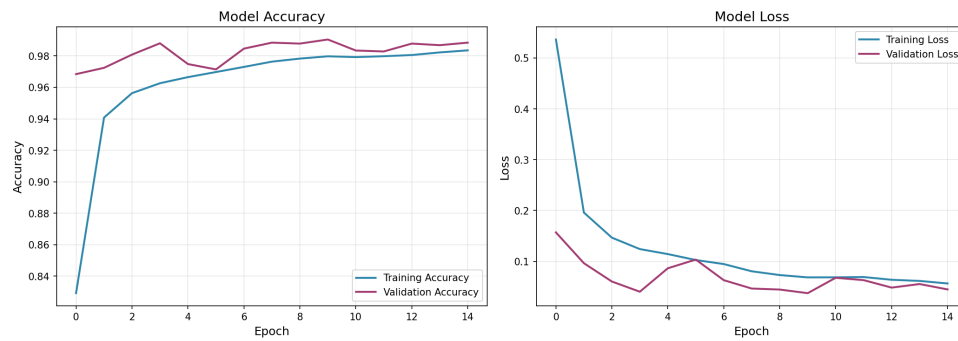


Figure 3: Training and validation accuracy/loss curves for digit recognition model

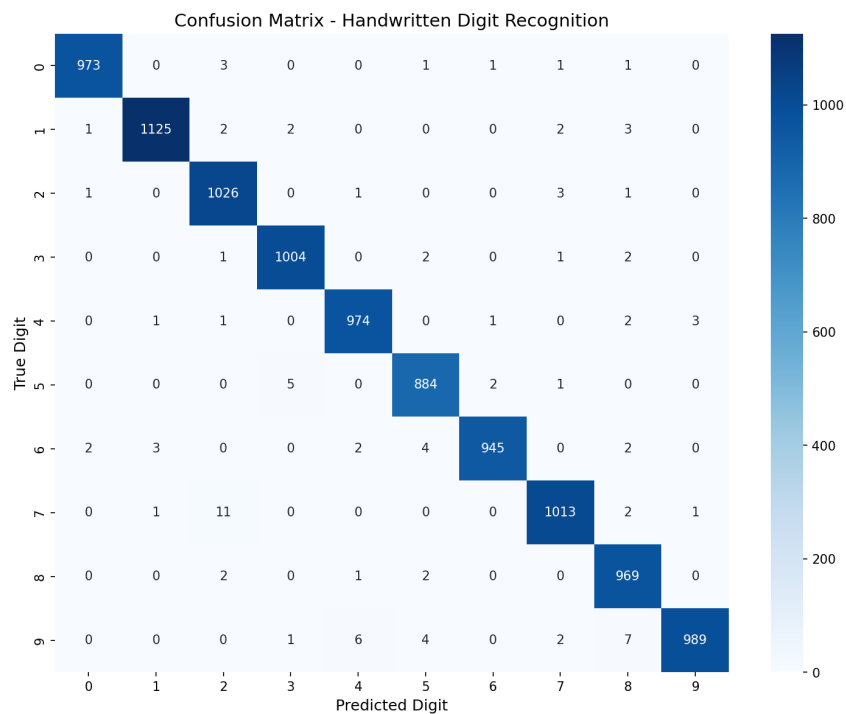


Figure 4: Confusion matrix for digit recognition on test set

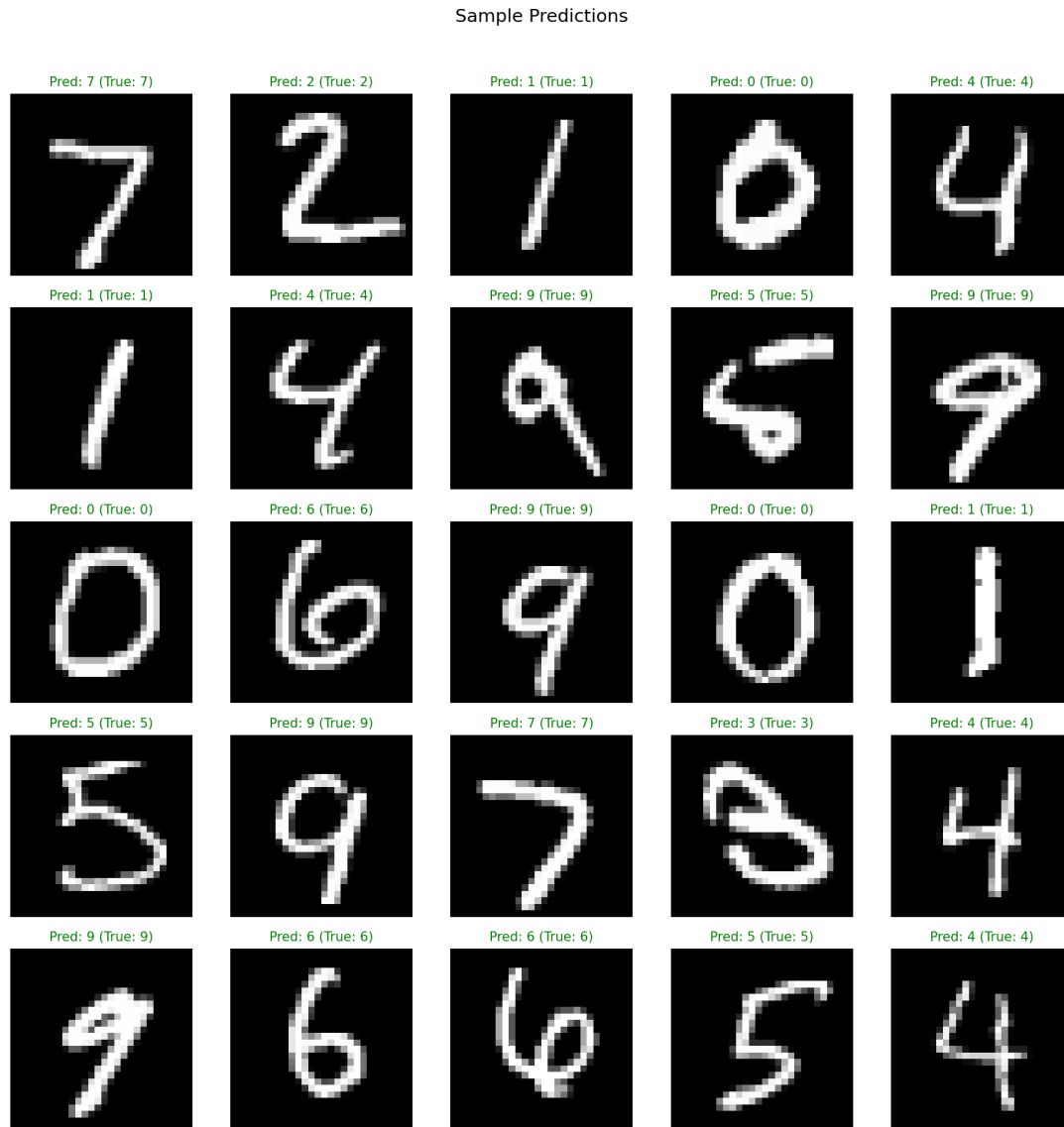


Figure 5: Sample predictions on test images

Table 4: Handwritten Digit Recognition Model Performance

Metric	Value
Test Accuracy	98.7%
Model Size (Float32)	245 KB
Model Size (Int8 Quantized)	72 KB
Inference Time (STM32F4)	~15 ms

3.7 TensorFlow Lite Conversion

Similar to the keyword spotting model, we apply full integer quantization:

```

1 def representative_dataset():
2     for i in range(1000):
3         yield [x_train[i:i+1].astype(np.float32)]
4 
```

```

5 converter = tf.lite.TFLiteConverter.from_keras_model(model)
6 converter.optimizations = [tf.lite.Optimize.DEFAULT]
7 converter.representative_dataset = representative_dataset
8 converter.target_spec.supported_ops = [
9     tf.lite.OpsSet.TFLITE_BUILTINS_INT8
10 ]
11 converter.inference_input_type = tf.int8
12 converter.inference_output_type = tf.int8
13 tflite_model = converter.convert()

```

Listing 2: TFLite Conversion for MNIST Model

3.8 STM32 Deployment

The deployment process involves:

- Converting TFLite model to C array using `xxd`
- Configuring TensorFlow Lite Micro runtime
- Implementing image capture interface
- Processing and displaying results via UART/LCD

4 Comparison and Discussion

Table 5: Comparison of Both Applications

Aspect	Keyword Spotting	Digit Recognition
Input Type	Audio (1D temporal)	Image (2D spatial)
Feature Extraction	Spectrogram	None (raw pixels)
Model Size (Quantized)	365 KB	72 KB
Test Accuracy	89.0%	98.7%
Inference Time	~50 ms	~15 ms
Complexity	Higher	Lower

4.1 Key Observations

1. The digit recognition model achieves higher accuracy with a smaller model size, primarily due to the simpler nature of the classification task.
2. Keyword spotting requires more complex feature extraction (MFCC), which adds to the overall processing time.
3. Both models fit comfortably within the memory constraints of STM32F4 series microcontrollers.
4. Quantization provides approximately 4× reduction in model size with minimal accuracy loss.

5 Conclusion

This homework successfully demonstrated the implementation of two embedded machine learning applications on STM32 microcontrollers. Both the keyword spotting and handwritten digit recognition systems achieve high accuracy while meeting the resource constraints of embedded systems.

Key takeaways include:

- Careful model architecture design is crucial for embedded deployment
- Quantization is essential for reducing model size without significant accuracy loss
- Feature extraction (like MFCC) can be performed on-device for real-time applications
- TensorFlow Lite for Microcontrollers provides a practical framework for embedded ML

The complete source code and trained models are available in the GitHub repository.

References

References

- [1] C. Ünsalan, B. Höke, and E. Atmaca, *Embedded Machine Learning with Microcontrollers: Applications on STM32 Boards*, Springer Nature, ISBN: 978-3031709111, 2025.
- [2] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv preprint arXiv:1804.03209, 2018.
- [3] Y. LeCun, C. Cortes, and C.J.C. Burges, "The MNIST Database of Handwritten Digits," 1998.
- [4] TensorFlow Team, "TensorFlow Lite for Microcontrollers," <https://www.tensorflow.org/lite/microcontrollers>.